Geospatial Indexing

turn two dimensions into cells you can search

Watch the 4-min video

In one line: ordinary indexes are one-dimensional, so spatial indexes map locations into **cells** (geohash prefixes, quadtree leaves, S2 or H3 cell IDs); a nearby search scans the target cell and its **neighbours**, then computes exact distances only for those few candidates.

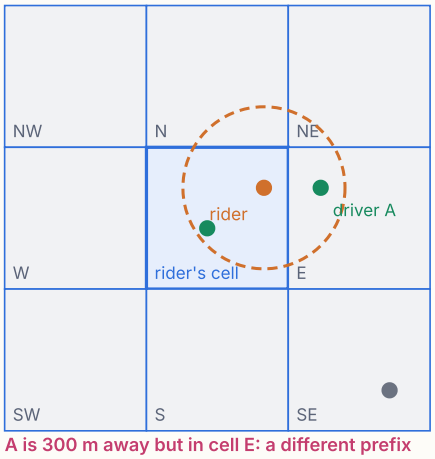
1 Nearest drivers, in milliseconds

A rider opens a ride-hailing app. In the city, 5,000 drivers report their location every 4 seconds (1,250 updates a second), and the app must find available drivers within 2 km in a few milliseconds. For one city you could almost scan them all; across hundreds of cities with millions of drivers and thousands of ride requests a second, comparing every rider with every driver does not scale. You need an index that answers "what is near this point?" directly.

2 Why ordinary indexes fail

A B-tree on `lat` finds a horizontal band around the whole planet; one on `lng` finds a vertical band. A query `WHERE lat BETWEEN … AND lng BETWEEN …` can use only one of them efficiently and must filter the band's millions of rows for the other. Spatial indexes solve this by turning 2D proximity into something a 1D structure can range-scan (a space-filling curve), or by building a tree over 2D regions.

3 Geohash



geohash: interleave longitude and latitude bits

lon bit, lat bit, lon bit … → base-32 string (Z-order curve)
length 5 ≈ 4.9 × 4.9 km · length 6 ≈ 1.2 × 0.6 km · 7 ≈ 153 m
shared prefix ⇒ same cell ⇒ nearby (usually)

```
WHERE geohash LIKE 'tdr1v%' → ordinary B-tree range scan
```

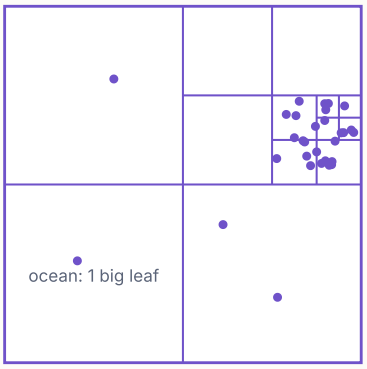
radius search done right

1 choose a length whose cell is ≥ the radius
2 search the rider's cell + its 8 neighbours
3 compute exact (haversine) distance, drop > 2 km
4 sort by distance, then by ETA on the road graph

Figure 1. Search the 3×3 block of cells around the rider, not just the rider's cell: points across a boundary can be close but share no prefix.

Geohash alternately halves longitude and latitude, writing one bit each time, and encodes the bits in base 32. Each extra character shrinks the cell about 32×. Because the bits follow a **Z-order curve**, nearby points *usually* share a prefix, so a prefix query is a range scan in any database or key-value store. The catch: two points either side of a cell boundary (or of the equator or the prime meridian) can have completely different prefixes, so always include the 8 neighbouring cells. Cells are also not equal in size: they shrink in width towards the poles.

4 Quadtrees and R-trees



quadtree: split a square into 4 when it holds > N points

N = e.g. 100; dense downtown → deep, tiny leaves; empty areas stay big
query: descend to the leaves that intersect the search circle
great for uneven, mostly static data (shops, restaurants)
moving points force splits and merges; usually rebuilt periodically

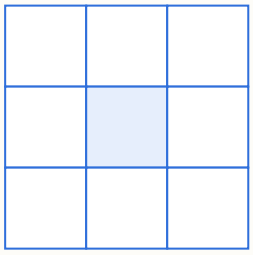
R-tree: nest bounding rectangles of objects

indexes shapes (roads, delivery zones), not just points

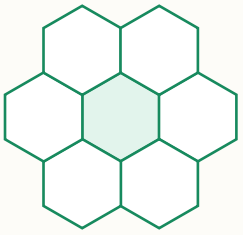
```
PostGIS GiST index: ST_DWithin(geom, point, 2000)
```

Figure 2. A quadtree adapts to density: the dense area in the top-right is split three times, the rest of the map not at all.

5 S2 and H3: modern cell systems



square: 4 neighbours at 1, 4 at 1.41



hexagon: all 6 neighbours at the same distance

H3 (Uber)

16 resolutions, 0–15
res 8 ≈ 0.74 km² per cell
gridDisk(cell, k) = k rings

S2 (Google): cube faces, Hilbert curve, 64-bit cell IDs, 31 levels (0–30)

Figure 3. Why hexagons: every neighbour is equally far away, so k rings of cells approximate a circle well.

S2 projects the sphere onto the six faces of a cube and orders cells along a Hilbert curve, which keeps neighbours closer in ID order than a Z-curve; any region can be covered by a small set of cell ID ranges. **H3** tiles the world with hexagons (plus 12 pentagons per resolution) at 16 resolutions; Uber uses it for dispatch, surge pricing and heat maps, because counting demand per equal-sized hexagon and taking "k rings" around a cell are simple and uniform.

Index	Cells	Strength	Weakness
Geohash	fixed grid, string prefix	works in any DB or KV store; simple	edge effects; cells distort near poles
Quadtree	adaptive squares	handles uneven density	in-memory structure; costly for fast-moving points
R-tree	bounding rectangles	shapes and polygons; PostGIS	updates rebalance the tree
S2	quad cells on a cube, Hilbert order	precise region coverings	more complex library
H3	hexagons, 16 resolutions	uniform neighbours; analytics	hexagons don't nest exactly into parents

6 Moving objects

Drivers move constantly, so the index lives in memory and is updated in place. **Redis GEO** commands store members in a sorted set whose score is a 52-bit interleaved geohash: `GEOADD drivers:pune`

`73.8567 18.5204 d-17` moves a driver in O(log n), and `GEOSEARCH drivers:pune FROMLONLAT 73.85 18.52 BYRADIUS 2 km ASC COUNT 20 WITHDIST` (Redis 6.2+) scans the covering cells and returns exact distances. Sorted-set members have no individual TTL, so stale drivers must be removed explicitly: keep the last update time in a second sorted set and `ZREM` drivers silent for 30 s, or rotate time-bucketed keys. Shard by city or region, since searches never cross a city boundary.

WORKED EXAMPLE: DRIVERS WITHIN 2 KM OF A RIDER

Each driver app sends its position every 4 s; the location service runs `GEOADD drivers:pune lon lat d-17` and records `ZADD drivers:pune:seen <now> d-17`; a sweeper removes drivers unseen for 30 s. A rider requests a ride: `GEOSEARCH … BYRADIUS 2 km ASC COUNT 50 WITHDIST` returns 50 candidates in about a millisecond. The service filters for *available* drivers with the right vehicle type, asks a routing service for ETAs on the road network (2 km straight-line can be 8 minutes across a river), and offers the ride to the best two or three.

TRADE-OFFS

- Smaller cells mean fewer false candidates but more cells to scan for a given radius.
- Geohash is portable and simple; S2/H3 give better geometry at the cost of a library dependency.
- Higher location update rates improve accuracy but multiply writes and battery use.
- Straight-line distance is cheap; road ETA is what matters, so compute it only for the final few candidates.

COMMON MISTAKES

- Searching only the rider's own geohash cell and missing drivers just across the boundary.
- Ranking by straight-line distance and ignoring rivers, one-way streets and traffic.
- Never removing stale positions, so offline drivers keep being offered rides.
- Treating latitude and longitude degrees as equal distances (a degree of longitude shrinks towards the poles).

7 Key terms

Geohash: base-32 string of interleaved lat/lng bits; prefix = cell

Quadtree: recursive 4-way split of space by density

S2 / H3: global cell systems: cube-based squares / hexagons

Neighbour cells: the 8 cells around a geohash cell, searched to fix edge effects

R-tree: tree of nested bounding rectangles, for shapes

Haversine: great-circle distance between two lat/lng points

8 Interview questions

Q1. How do you find nearby points efficiently? BASIC

Map locations into cells with a spatial index, search the target cell and its neighbours, then compute exact distances only for those candidates.

Q2. Geohash vs quadtree? BASIC

Geohash is a fixed grid encoded as string prefixes, works in any database but has edge effects; a quadtree splits adaptively by density, better for uneven data.

Q3. Design 'restaurants near me' for 1 million static restaurants. SCENARIO

Store restaurants with a geohash or use PostGIS (GiST) and ST_DWithin; cache popular cells; data changes rarely, so a quadtree or R-tree works well.

Q4. How do you keep driver locations fresh? SCENARIO

Update every few seconds with GEOADD, record last-seen times, sweep out drivers silent for ~30 s, and shard the index by city.

Q5. Why are hexagons popular for ride-hailing analytics? TRICKY

All six neighbours are equidistant and cells are similar in size, so rings approximate circles and per-cell counts (demand, supply) compare fairly.

Q6. Two points 50 m apart have geohashes with no common prefix. How? TRICKY

They sit either side of a major cell boundary (e.g. the equator or prime meridian); the Z-order curve jumps there, which is why neighbour cells are searched.

Next: SD-Q-012 · Leaderboards backendroadmap.com